

Unit 3 — Logic & Prolog

Review deck for the test on Thu Oct 8

English for the exam · "ES:" tips in Spanish

Wiki: Propositional & First-Order Logic · Horn Clauses · Unification · Prolog · Recursion & Lists

Propositional vs. first-order logic

	Propositional	First-order (FOL)
Atoms	symbols: true / false	objects, predicates, functions
Quantifiers	none	$\forall$ (for all), $\exists$ (exists)
Can say "every student..."	no	yes
Decidable?	yes	no — only semi-decidable

CNF (conjunctive normal form) = AND of ORs of literals; every propositional sentence can be converted. It is the input format for resolution.

CNF conversion recipe

1. Eliminate $\Leftrightarrow$: $A \Leftrightarrow B \rightarrow (A \Rightarrow B) \wedge (B \Rightarrow A)$
2. Eliminate $\Rightarrow$: $A \Rightarrow B \rightarrow \neg A \vee B$
3. Push $\neg$ inward (De Morgan, double negation)
4. Distribute $\vee$ over $\wedge$

Practice 1:

- $A \Leftrightarrow B \rightarrow (\neg A \vee B) \wedge (\neg B \vee A)$
- $(A \Rightarrow B) \Rightarrow C \rightarrow (A \wedge \neg B) \vee C \rightarrow (A \vee C) \wedge (\neg B \vee C)$
- $\neg(A \wedge (B \vee C)) \rightarrow (\neg A \vee \neg B) \wedge (\neg A \vee \neg C)$

Horn clauses

A **Horn clause** has **at most one positive literal**.

Kind	Positive literals	Logic	Prolog
Definite clause (rule)	exactly 1	$\neg A \vee \neg B \vee C \equiv A \wedge B \Rightarrow C$	<code>c :- a, b.</code>
Fact	1, empty body	C	<code>c.</code>
Goal clause (query)	0	$\neg A \vee \neg B$	<code>?- a, b.</code>
Not Horn	≥ 2	$A \vee B$	—

Why it matters: inference by **forward / backward chaining**; propositional entailment is **linear** in the size of the KB.

Prolog = logic + control

- Prolog = **first-order definite clauses + SLD resolution**: backward chaining, **depth-first, left to right**, with unification and backtracking.
- "**Algorithm = Logic + Control**" (Kowalski): you write the logic; Prolog supplies the control → **clause order and goal order matter**.
- Created by **Colmerauer & Roussel, Marseille, 1972** (Kowalski's theory).
Declarative.

Four syntax conventions: implication written backwards (`c :- a, b.`) · **Uppercase = variable** · variables implicitly $\forall$ · `,` = and, `;` = or, every clause ends with `.`

family.pl (lecture example)

```
parent(hector, ana).    parent(hector, luis).
parent(ana, sofia).     parent(luis, diego).

grandparent(X, Z) :- parent(X, Y), parent(Y, Z).
sibling(X, Y)      :- parent(P, X), parent(P, Y), X \= Y.
ancestor(X, Y)     :- parent(X, Y).
ancestor(X, Y)     :- parent(X, Z), ancestor(Z, Y).
```

Query	Answer
?- parent(hector, X).	X = ana ; X = luis.
?- findall(Z, grandparent(hector, Z), L).	L = [sofia, diego].
?- findall(D, ancestor(hector, D), L).	L = [ana, luis, sofia, diego].

Unification

Find a substitution that makes two terms identical, or **fail**. Two-way pattern matching.

Query	Result
$p(X, f(Y)) = p(a, f(b))$	$\{X/a, Y/b\}$
$p(X, X) = p(a, b)$	fail
$f(X, g(X)) = f(Y, g(a))$	$\{X/a, Y/a\}$
$[H T] = [a]$	$\{H/a, T/[]\}$
$[A, B] = [1, 2, 3]$	fail (different lengths)
$X = 2 + 3$	$X = 2+3$ — a term , not 5

ES: Prolog omite el *occurs check*: $X = f(X)$ crea un término cíclico.

Arithmetic: `=` vs `is` vs `==`

Query	Result	Why
<code>X = 2 + 3.</code>	<code>X = 2+3</code>	unification, no evaluation
<code>X is 2 + 3.</code>	<code>X = 5</code>	evaluates the right side
<code>2 + 3 == 5.</code>	<code>true</code>	compares values
<code>2 + 3 = 5.</code>	<code>false</code>	different structures
<code>X is Y + 1.</code> (Y unbound)	error	"Arguments are not sufficiently instantiated"

ES: en una regla recursiva, `S is S0 + H` va **después** de la llamada que liga `S0`.

SLD trace: factorial(2, X)

```
factorial(0, 1).  
factorial(A, B) :- A > 0, C is A - 1, factorial(C, D), B is A * D.  
  
factorial(2, X): clause 1 fails; clause 2: 2 > 0, C = 1, call factorial(1, D1)  
  factorial(1, D1): clause 1 fails; clause 2: 1 > 0, C = 0, call factorial(0, D2)  
    factorial(0, D2): clause 1 → D2 = 1    (choice point left for clause 2)  
  D1 is 1 * 1 = 1  
X is 2 * 1 = 2      → X = 2 ;  
backtrack: factorial(0, D2) via clause 2 fails (0 > 0 false) → false.
```

Negation as failure & the cut

- `\+ G` succeeds if G **cannot be proved** → logical negation only under the **closed-world assumption**.
- Only negate goals whose variables are **bound**: `\+ parent(X, ana)` is `false` (some X exists).
- **Cut** `!` commits to the choices made so far in the clause.

```
% buggy:  ?- max(5, 3, 3).  → true  (wrong!)
max(X, Y, X) :- X >= Y, !.
max(_, Y, Y).
% fix 1: unify the output AFTER the cut
max(X, Y, M) :- X >= Y, !, M = X.
max(_, Y, Y).
% fix 2 (better): no cut
max(X, Y, M) :- ( X >= Y -> M = X ; M = Y ).
```

Recursion, lists and left recursion

```
last(X, [X]).  
last(X, [_|T]) :- last(X, T).  
  
sum_list([], 0).  
sum_list([H|T], S) :- sum_list(T, S0), S is S0 + H.
```

- Base case first; each recursive call must make progress.
- **Accumulators** turn exponential/stack-heavy recursion into linear tail recursion (Fibonacci).
- **Left recursion loops:** `path(X,Z) :- path(X,Y), link(Y,Z).` → stack overflow.
Fix: `path(X,Z) :- link(X,Y), path(Y,Z).` or `:- table path/2.`

Collecting solutions

Predicate	Duplicates	No solutions
<code>findall(T, G, L)</code>	kept	<code>L = []</code>
<code>setof(T, G, L)</code>	sorted, removed	fails (use <code>Var^Goal</code>)
<code>bagof(T, G, L)</code>	kept, grouped by free vars	fails
<code>aggregate_all(count, G, N)</code>	—	<code>N = 0</code>

ES: errores de tu Lab 01 a recordar: `spouse/2` asimétrico (B4) y avisos *choicpoint* (B5); `isPerm` de clase es igualdad de conjuntos, no permutación (B7).

Self-check (answer aloud, in English)

1. Is $\neg A \vee B \vee C$ a Horn clause? Why?
2. What does `X = 2 + 3` return, and why is it not 5?
3. Unify `f(X, g(X))` with `f(Y, g(a))`.
4. Why does `\+ parent(X, ana)` fail?
5. Why does a left-recursive `path/2` loop forever?
6. Explain "Algorithm = Logic + Control".

Answers: wiki `study/practice-unit-3-logic.md` · Exam Drill quiz